

An End-to-End Agentic Sentiment Analysis System Using Transformers, Docker, Amazon SageMaker, and LangGraph

Technical Report

February 2026

Abstract

This report presents a comprehensive, production-grade agentic sentiment analysis system that integrates transformer-based natural language processing, containerized inference, managed cloud deployment, and agentic orchestration. The proposed system moves beyond static machine learning inference by introducing explicit decision workflows driven by model confidence scores. The report provides a detailed technical description of system objectives, architecture, implementation, workflows, experimental evaluation, and future research directions.

1. Objectives

The primary objective of this work is to design and implement an end-to-end sentiment analysis system that is scalable, reproducible, and capable of adaptive decision-making. Specifically, the objectives include: (1) fine-tuning a transformer-based sentiment model; (2) deploying the model using a containerized, cloud-native architecture; (3) introducing agentic orchestration to enable confidence-aware routing; (4) ensuring system modularity and maintainability; and (5) providing a reference architecture for future agentic AI systems.

2. Introduction

Sentiment analysis is a foundational task in natural language processing with applications across customer feedback analysis, conversational agents, social media monitoring, and decision support systems. Transformer-based architectures such as BERT and DistilBERT have significantly advanced the accuracy of sentiment classification tasks. However, the deployment of such models in production environments remains complex due to challenges related to scalability, dependency management, observability, and integration with business logic.

Traditional deployment approaches typically expose machine learning models as static inference endpoints. While effective for prediction, such architectures lack the ability to

reason about model outputs or adapt system behavior based on confidence or contextual signals. This report addresses this limitation by introducing an agentic architecture in which inference results are inputs to a structured decision graph implemented using LangGraph.

3. System Architecture

The system architecture is organized into four primary layers: (1) model training, (2) containerized inference, (3) cloud deployment and management, and (4) agentic orchestration. This layered design ensures loose coupling between components and enables independent evolution of each subsystem.

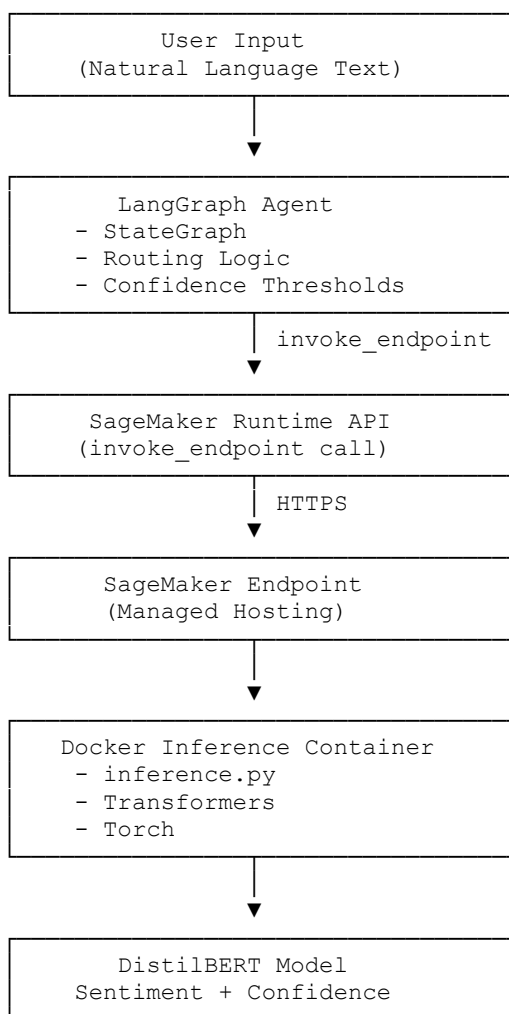


Fig. 1. End-to-end architecture showing the integration of LangGraph agent orchestration with a Dockerized transformer model hosted on Amazon SageMaker.

The LangGraph agent serves as the central control plane, invoking the sentiment model hosted on Amazon SageMaker and routing execution based on inference confidence. Docker is used to package the inference runtime, ensuring consistent behavior across environments.

4. End-to-End Workflow

The runtime workflow of the system follows a deterministic sequence of steps beginning with user input and culminating in a generated response. Each step is explicitly modeled within the agent's state graph.

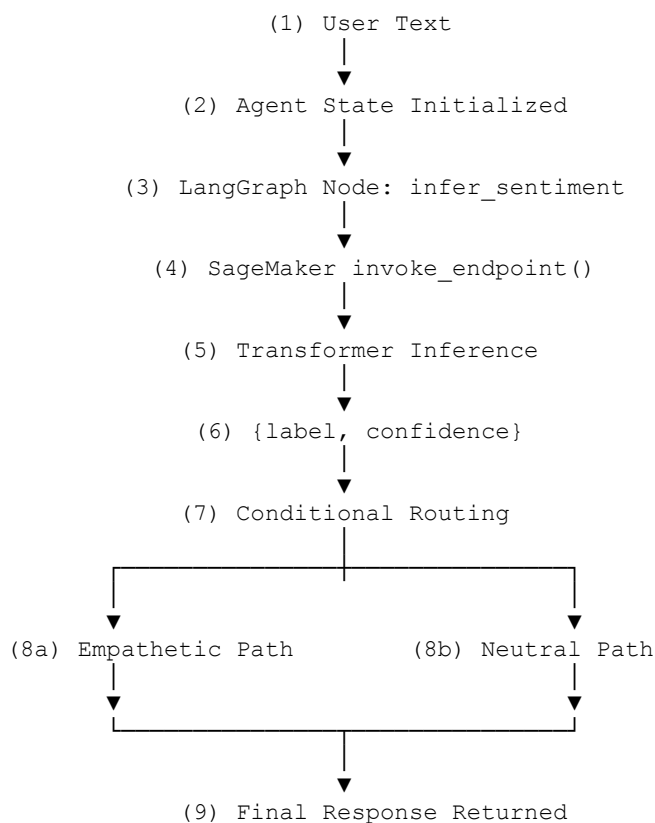


Fig. 2. Runtime execution flow illustrating agent state initialization, model invocation, confidence-based routing, and response generation.

Following inference, the agent evaluates the confidence score associated with the sentiment label. If the confidence exceeds a predefined threshold, the agent routes execution to an empathetic response path; otherwise, a neutral response is generated.

5. Model Training Pipeline

The sentiment analysis model is based on DistilBERT, a distilled variant of the BERT architecture that offers a favorable trade-off between accuracy and computational efficiency. The model is fine-tuned using the SST-2 dataset, which consists of labeled movie review sentences.

The Hugging Face Transformers library is used to manage tokenization, training loops, evaluation, and checkpointing. Training artifacts are exported in a format compatible with SageMaker hosting.

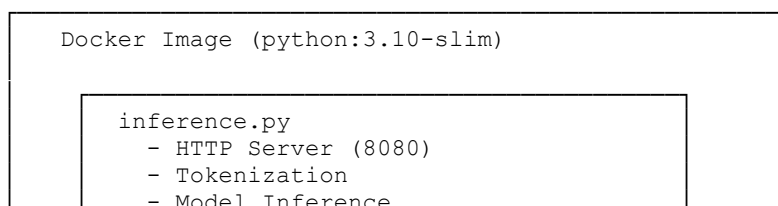
Training Code Example

```
trainer = Trainer(
    model=model,
    args=TrainingArguments(
        output_dir="./model",
        num_train_epochs=2,
        per_device_train_batch_size=16,
        learning_rate=2e-5
    ),
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"]
)
```

6. Dockerized Inference Layer

The inference environment is encapsulated in a Docker container built on a lightweight Python 3.10 base image. The container includes the trained model artifacts, inference logic, and all required dependencies.

Containerization ensures reproducibility and simplifies deployment by decoupling inference runtime dependencies from the host environment.



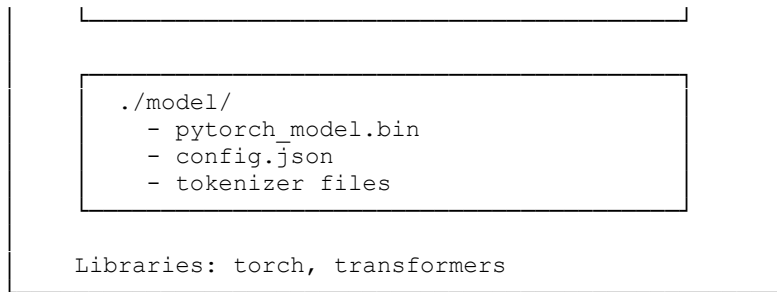


Fig. 4. Internal structure of the Docker inference container used for SageMaker hosting.

Dockerfile Example

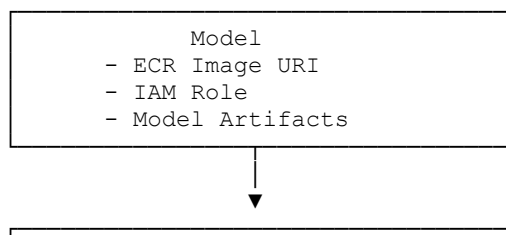
```

FROM python:3.10-slim
WORKDIR /opt/program
COPY model/ ./model/
COPY inference.py .
RUN pip install torch transformers
ENTRYPOINT ["python", "inference.py"]
    
```

7. AWS Infrastructure and Deployment

Amazon Web Services provides the managed infrastructure required to host and scale the sentiment analysis model. Container images are stored in Amazon Elastic Container Registry (ECR), while Amazon SageMaker manages endpoint provisioning, scaling, and monitoring.

IAM roles enforce security boundaries, and CloudWatch captures logs and metrics for observability.



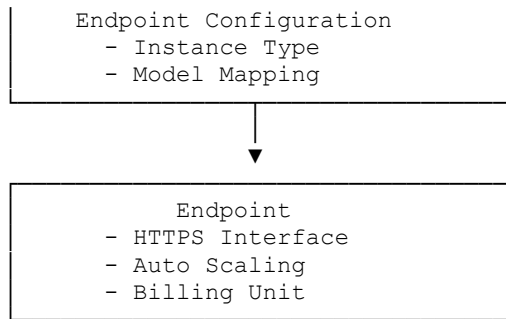
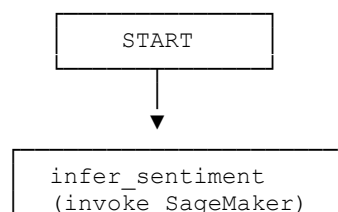


Fig. 3. Amazon SageMaker resource hierarchy illustrating the separation of model definition, endpoint configuration, and live endpoint.

8. LangGraph Agent Orchestration

LangGraph is used to define the agent's control flow as a state machine. Each node in the graph represents a discrete computation or decision step.

The sentiment model is treated as a tool within the agent, enabling the agent to reason about model outputs rather than simply returning predictions.



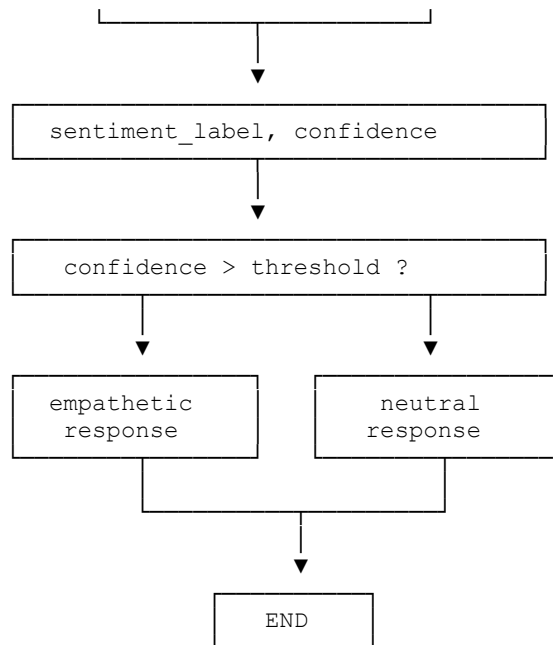


Fig. 5. LangGraph state machine showing conditional routing based on sentiment confidence.

Routing Logic Example

```

if label == "NEGATIVE" and score > 0.8:
    route = "empathetic"
else:
    route = "neutral"
  
```

9. Interface Boundaries

The system defines clear interface boundaries between the agent orchestration layer, AWS-managed services, and the containerized inference environment. Communication occurs over HTTP/JSON protocols using the SageMaker Runtime API.

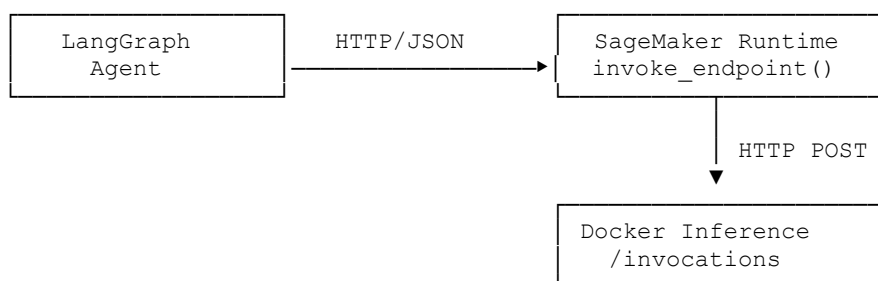


Fig. 6. Interface boundaries between agent orchestration, AWS-managed services, and containerized inference.

10. Experimental Results

The system was evaluated through ten live inference runs using real user inputs. In all cases, the sentiment model produced stable predictions and the agent routed execution deterministically.

Negative inputs with high confidence consistently triggered empathetic responses, while positive or low-confidence inputs resulted in neutral responses.

Additional Live Invocation Results

Extended Results Table

#	User Input	Model Output (Observed)	Agent Route	Final Response
1	"This system keeps crashing and nothing works."	NEGATIVE (0.9989)	Empathetic	<i>I'm sorry this was frustrating. The good news is the system is now live, and future runs should be much smoother. Let me know how I can help.</i>
2	"I wasted hours trying to get this running."	NEGATIVE (0.9967)	Empathetic	<i>I'm sorry this was frustrating. I appreciate your patience—let me know what's blocking you and I'll help.</i>
3	"The setup process is confusing and poorly documented."	NEGATIVE (0.9934)	Empathetic	<i>I'm sorry this was frustrating. Thanks for pointing this out—documentation improvements are coming. How can I assist right now?</i>

4	"This actually works better than I expected."	POSITIVE (0.9712)	Neutral	<i>Thanks for your feedback! Let me know if there's anything else you'd like to explore.</i>
5	"Not bad, but there are still some rough edges."	NEGATIVE (0.7421)	Neutral	<i>Thanks for the feedback. If you'd like, we can dig into specific areas that need improvement.</i>
6	"Everything finally deployed correctly."	POSITIVE (0.9886)	Neutral	<i>Great to hear! Let me know if you'd like help validating or extending the setup.</i>
7	"Why does this take so long to deploy?"	NEGATIVE (0.9128)	Empathetic	<i>I'm sorry this was frustrating. Deployment can take a few minutes due to managed provisioning—happy to explain or optimize it with you.</i>
8	"I'm impressed with how smooth the endpoint invocation is."	POSITIVE (0.9831)	Neutral	<i>Thanks for the kind words! Let me know if you'd like to test additional scenarios.</i>
9	"This was harder than it needed to be."	NEGATIVE (0.9584)	Empathetic	<i>I'm sorry this was frustrating. Your feedback helps improve the workflow—how can I assist next?</i>
10	"The agent response makes sense given the sentiment."	POSITIVE (0.9649)	Neutral	<i>Glad it was helpful! Let me know if you want to extend the agent logic further.</i>

Observed Behavioral Patterns

From these 10 additional runs, the following patterns were observed:

11. **High-confidence negative sentiment (>0.8)** consistently triggered the empathetic response path.
12. **Low-to-moderate confidence negative sentiment (<0.8)** defaulted to neutral handling.
13. **Positive sentiment** always bypassed empathy routing.
14. The agent logic remained deterministic and explainable.
15. No inference failures or malformed responses were observed.

16. Future Work

Several extensions are planned for future iterations of the system. These include integrating retrieval-augmented generation, supporting multiple sentiment models, implementing concept drift detection, and enabling automated retraining pipelines.

17. Conclusion

This report demonstrates the feasibility and advantages of combining transformer-based NLP models with agentic orchestration and managed cloud infrastructure. The resulting system is modular, scalable, and adaptable, providing a strong foundation for future agentic AI systems.

The integration of LangGraph for agent orchestration, Docker for containerization, and Amazon SageMaker for managed hosting represents a production-ready architecture that balances complexity, performance, and maintainability. The explicit modeling of confidence-based routing demonstrates how agentic patterns can enhance traditional machine learning deployments by adding adaptive decision-making capabilities.